

Rapport de projet

Drone Instrumenté Piloté par IA

License Professionnelle MECSE 2024-2025

Professeurs référents : Mr.Mininger et Mr.Clamens



Djoudi Rayan

Sakho Bouna

Galeota Baptiste

Boutkayoud Abderrahmane



Table des matières

Présentation du projet.....	3
Cahier des charges.....	4
Synoptique.....	4
Carte mentale.....	5
Diagrammes SysML.....	5
Diagramme des exigences.....	5
Diagramme de cas d'utilisation.....	6
Diagramme de blocs.....	6
Composants.....	7
STM32F303.....	7
Capteur GPS.....	7
Capteur de son.....	7
Bluetooth HC-05.....	8
Parrot Bebop 2.....	8
Répartition des tâches.....	9
Diagramme de Gantt.....	9
Gestion du drone.....	10
Présentation de la partie.....	10
Conception de la carte.....	10
Schéma structurel.....	10
Typon.....	11
Carte.....	11
Instrumentation capteurs.....	12
Présentation de la partie.....	12
Objectifs.....	12
Gestions des périphériques.....	12
Développement logiciel.....	13
Tests.....	15
Développement API.....	17
Présentation de la partie.....	17
Programme.....	18
Diagramme de classe.....	24
Diagramme de classe simplifié.....	24
Tests.....	25
Pilotage du drone avec IA.....	27
Développement du Système de Contrôle du Drone par Reconnaissance de Gestes... 27	27
1. Introduction et Objectifs.....	27
2. Méthodologie de Développement.....	27
3. Phase 1 : Détection de Gant sur Images Statiques.....	28
4. Phase 2 : Acquisition du Flux Vidéo Drone.....	29
5. Phase 3 : Fusion et Tests Préliminaires.....	30
6. Phase 4 : Optimisation du Pipeline Vidéo.....	31



7. Phase 5 : Intégration de l'Intelligence Artificielle.....	33
8. Résultats et Performances.....	38
9. Défis Techniques Surmontés.....	39
10. Interface Utilisateur et Expérience.....	40
11. Technologies et Outils Utilisés.....	40
12. Limitations et Améliorations Futures.....	42
13. Conclusion.....	43
Conclusion.....	44



Présentation du projet

Dans le cadre de notre Licence Professionnelle MECSE, nous avons réalisé un projet de fin d'année en équipe de quatre, combinant électronique embarquée, développement mobile et intelligence artificielle. L'objectif principal était d'instrumenter un drone avec des capteurs afin de collecter des données, gps et de niveau sonore, dans des zones difficiles d'accès.

Ces données sont ensuite transmises et affichées en temps réel sur une application Android que nous avons développée. En parallèle, nous avons mis en place un système de pilotage du drone par gestes, en utilisant une intelligence artificielle entraînée à partir d'images pour reconnaître les mouvements de l'utilisateur. Ce projet nous a permis de mettre en pratique nos compétences pluridisciplinaires acquises au cours de l'année et de travailler ensemble autour d'un défi technologique.

Lien répertoire GitHub

https://github.com/Letsuchinoko/Projet_Drone





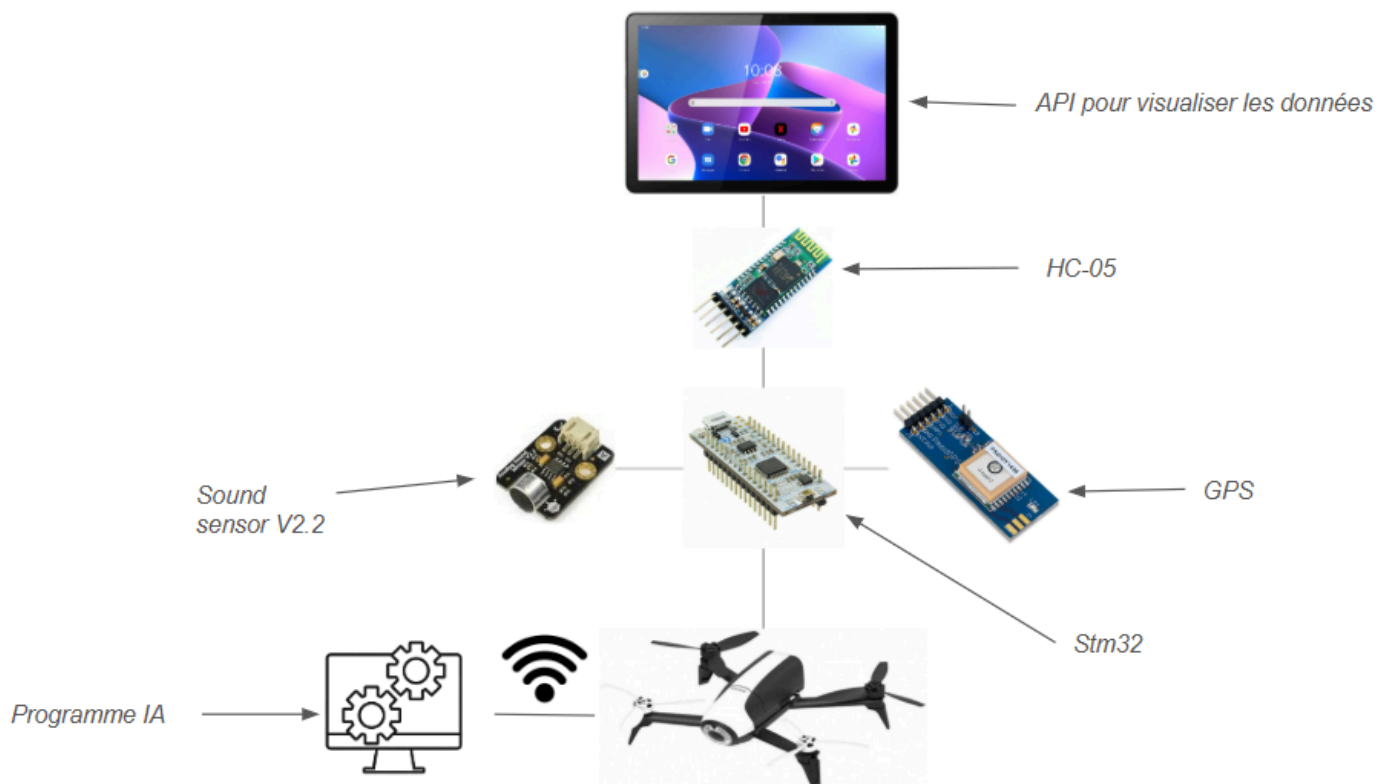
Cahier des charges

Objectif 1 : Instrumenter un drone afin de récupérer des informations de capteurs déposés sur des lieux difficilement atteignables, afficher les infos sur IHM Android.

Objectif 2 : Piloter le drone à l'aide de gestes effectués par l'utilisateur, grâce à l'entraînement d'une IA.

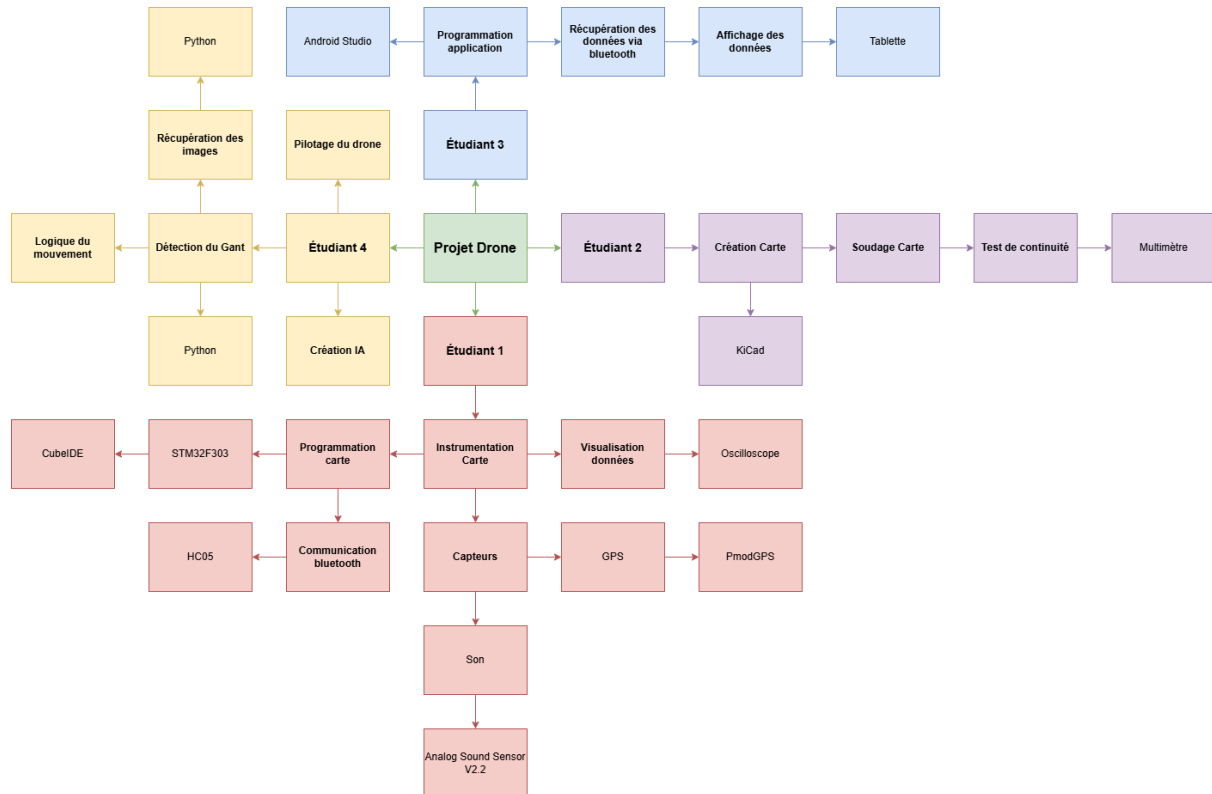
- Instrumentation du drone avec capteurs
- Développer une API Android avec affichage des données .
 - Récupération des informations de gestes effectués par l'utilisateur avec une IA entraînée sur des photos.

Synoptique





Carte mentale



Diagrammes SysML

Diagramme des exigences

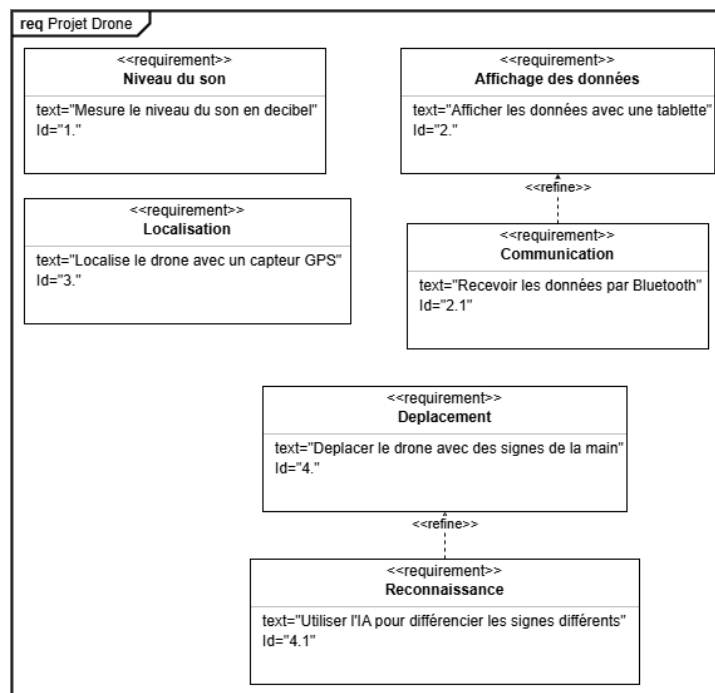




Diagramme de cas d'utilisation

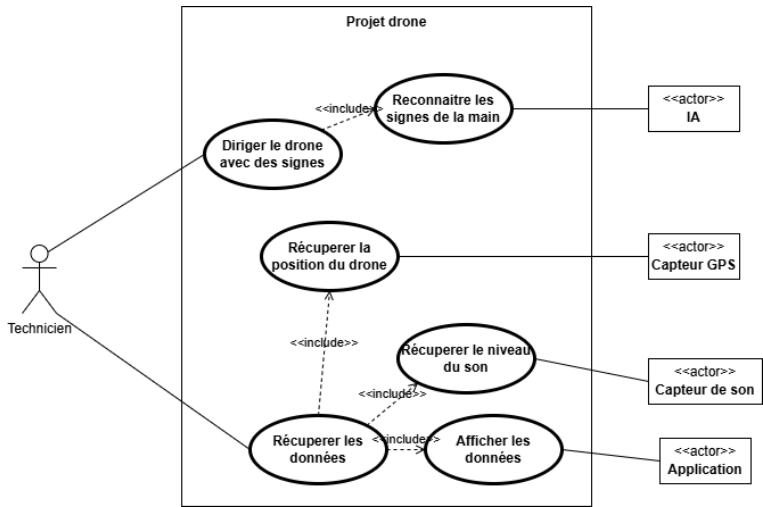
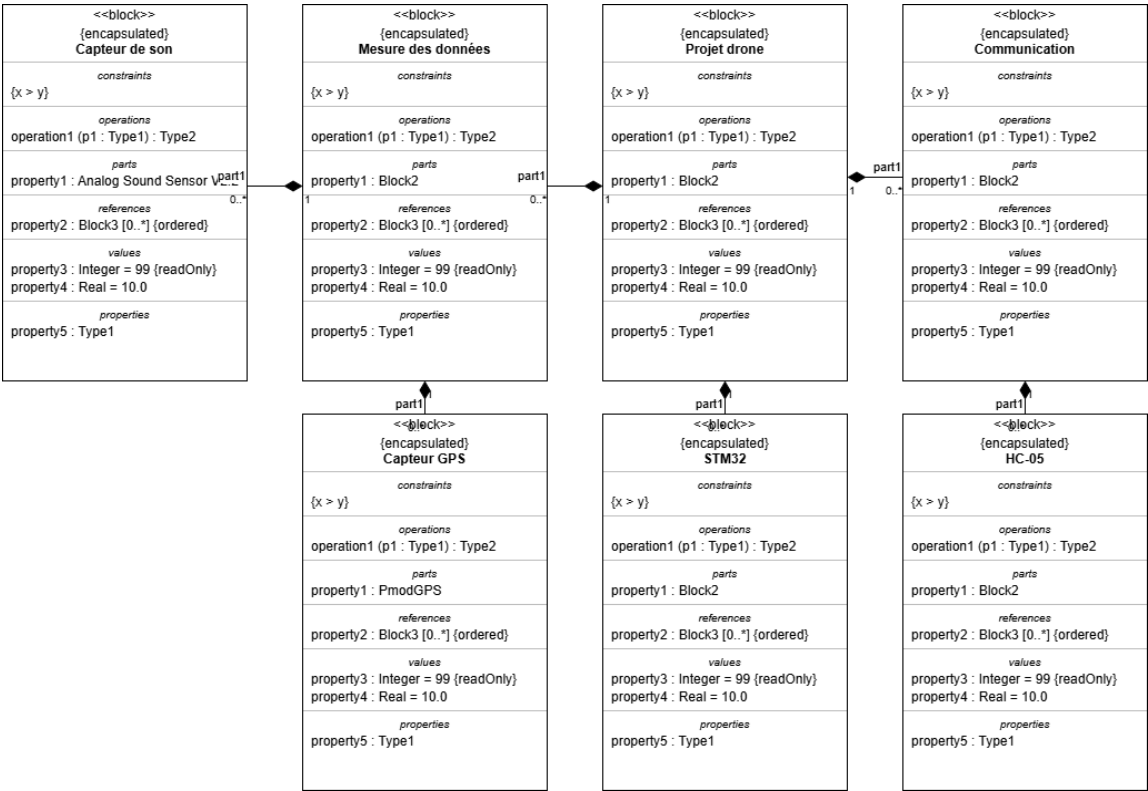


Diagramme de blocs





Composants

STM32F303



Le STM32F303 est un microcontrôleur de la série des STM32 Nucleo.
Il est alimenté en 5V.
Il possède 32 broches dont 8 broches analogues et 14 broches numériques.

Capteur GPS



Le Pmod GPS est un module conçu pour être utilisé avec une carte Arduino, il intègre un capteur GPS et une antenne.
Il a besoin d'une alimentation 3.3V.

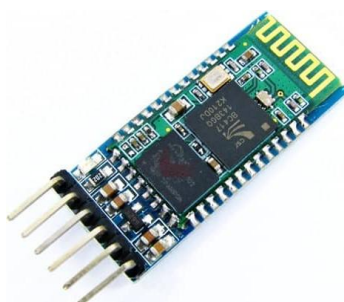
Capteur de son



L'Analog Sound Sensor est un module conçu pour être utilisé avec une carte Arduino, il intègre un capteur de son.
Il peut être alimenté avec une tension entre 3.3V et 5V.



Bluetooth HC-05



Le HC-05 est un module Bluetooth sans fil.

Il peut être alimenté avec une tension entre 3.3V et 5V.

Il peut communiquer avec une vitesse de 9600 bauds jusqu'à 115200 bauds.

Parrot Bebop 2



Le Bebop 2 est un drone civil à quatre rotors du constructeur français Parrot, il est le successeur du Bebop.

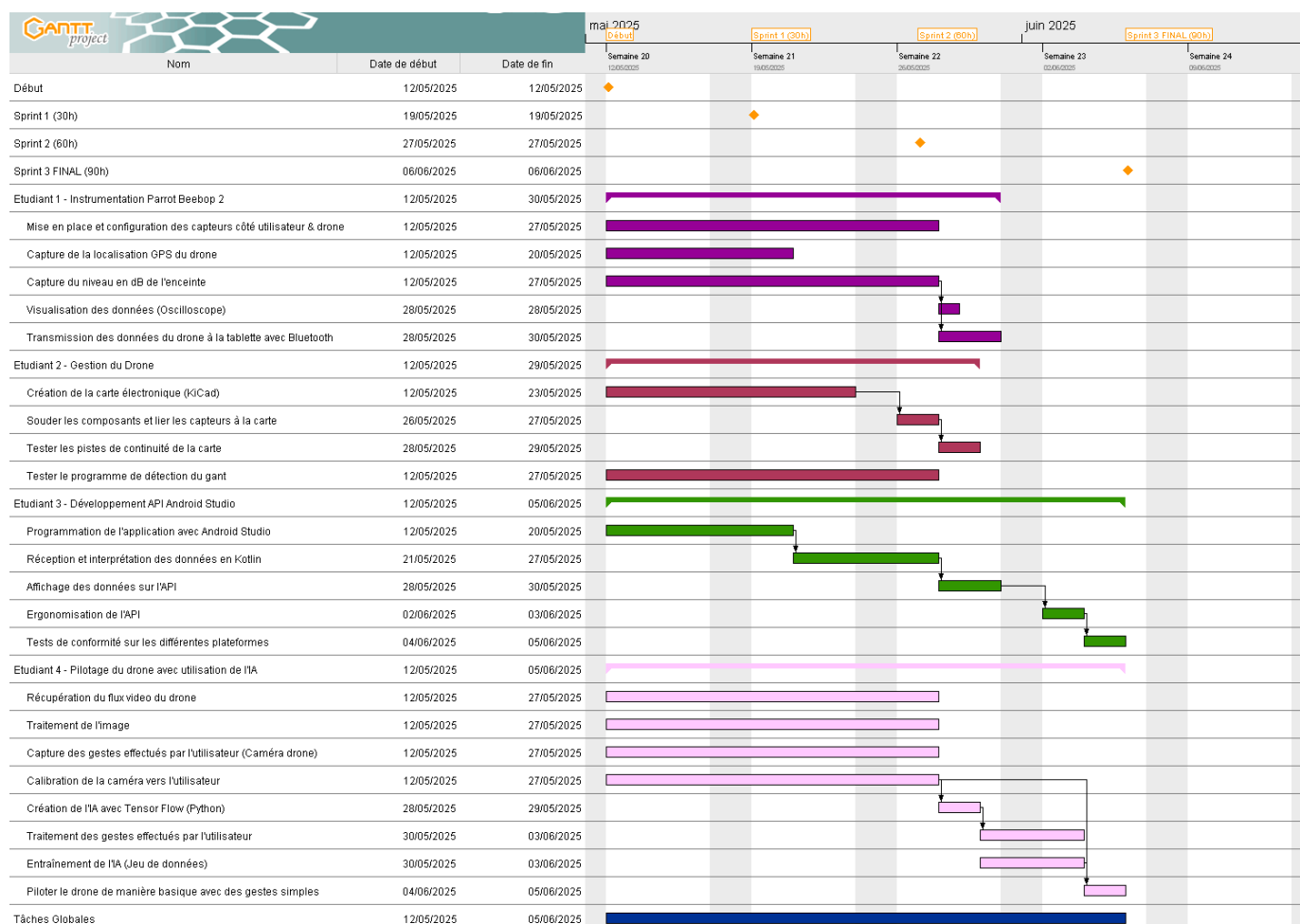
Il a une autonomie de 25 minutes.



Répartition des tâches

Nom	Fonction
Galeota Baptiste	Gestion du drone
Sakho Bouna	Instrumentation Capteurs
Boutkayoud Abderrahmane	Développement API
Djoudi Rayan	Pilotage du drone avec IA

Diagramme de Gantt





Gestion du drone

Présentation de la partie

Cette partie consiste à créer une carte électronique pour connecter les capteurs au micro-contrôleur.

Conception de la carte

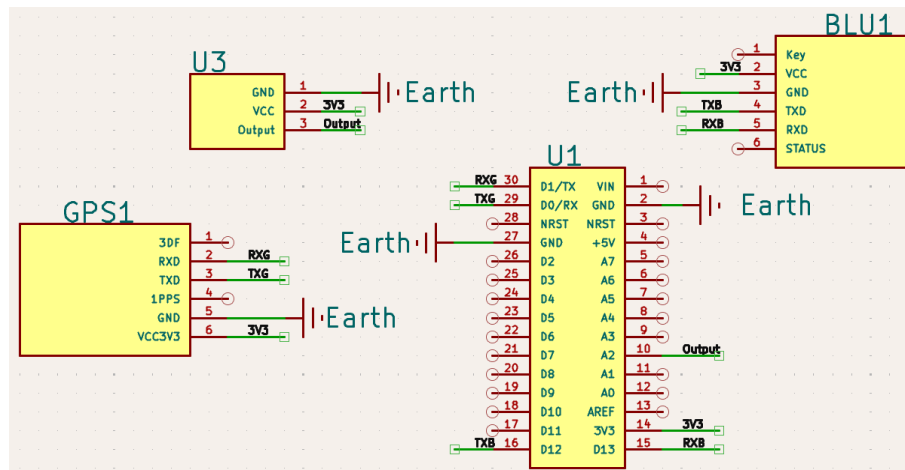
Schéma structurel

U1=STM32

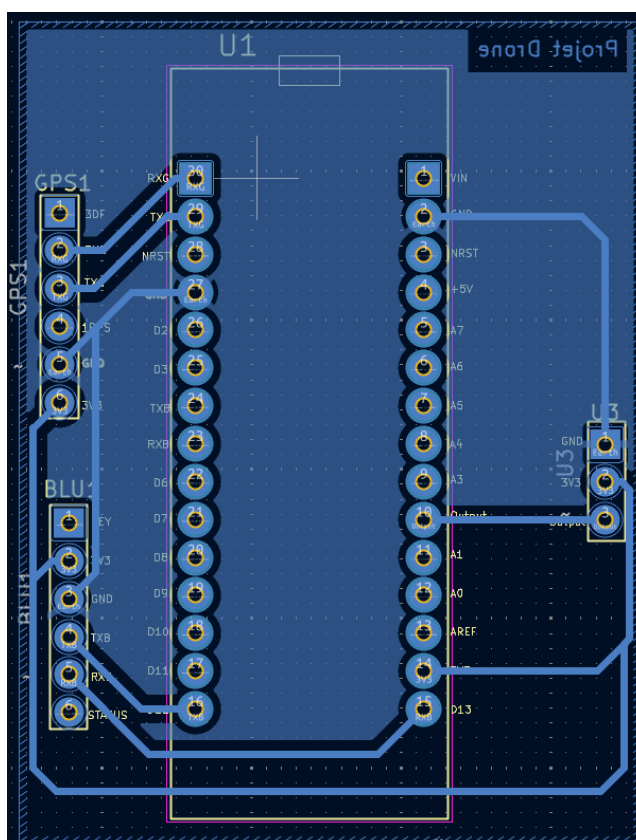
U3=Capteur de son

GPS1=Capteur GPS

BLU1=Module Bluetooth HC-05

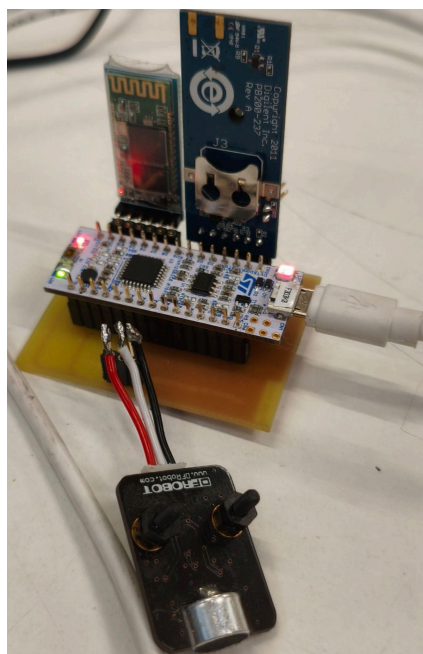


Typon



*Lors de la création de la carte, j'ai essayé de prendre le moins de place possible.
À part celle de la carte, j'ai créé les empreintes moi-même.*

Carte



Le rendu final de la carte avec les composants déposés.



Instrumentation capteurs



Présentation de la partie

Dans le cadre du projet de groupe, mon rôle a été la conception, la programmation et la mise en œuvre de la partie embarquée sur microcontrôleur STM32. Cette partie vise à acquérir des données de capteurs (GPS, capteur sonore), à les traiter, puis à les transmettre pour exploitation sur une interface Android. Cette partie détaille les étapes, choix techniques, difficultés rencontrées et résultats obtenus.

Objectifs

Les principaux objectifs de ma tâche consistent en 3 sous-tâches que j'ai décomposées ci dessous

- Acquisition de la localisation GPS du drone, plus précisément des données importantes à communiquer à l'utilisateur telles que la latitude, la longitude et l'altitude qui permettront de connaître la position et la hauteur du drone.
- Mesure du niveau sonore (en dB) qui sera détectée puis transformée afin d'avoir un aperçu en décibels, nous aurions pu décider d'afficher la valeur numérique brute convertie mais dans un cadre à destination d'utilisateur il est préférable d'utiliser une unité de mesure plus appréhensible.
- Transmission des données vers l'interface utilisateur

Gestions des périphériques

J'ai utilisé l'UART pour la transmission de données car il s'agit d'un protocole simple, universel et directement supporté par la plupart des modules externes (GPS, Bluetooth, etc.). Il permet une communication fiable entre le microcontrôleur et les périphériques ou interfaces utilisateurs, tout en étant facile à mettre en œuvre sur STM32 grâce aux périphériques matériels dédiés.

Toute la programmation du matériel listée plus haut a été faite via le logiciel stm32Cube IDE qui est amplement suffisante pour le pilotage des appareils utilisés.

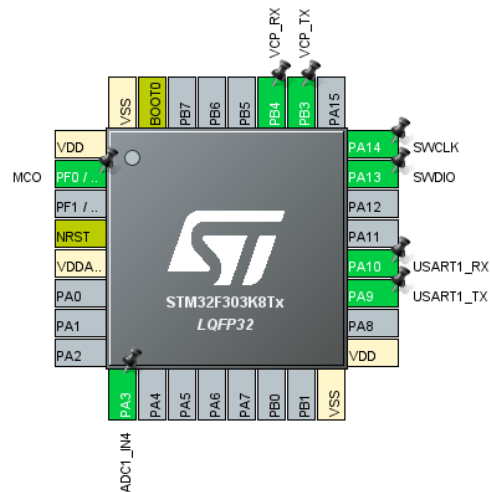


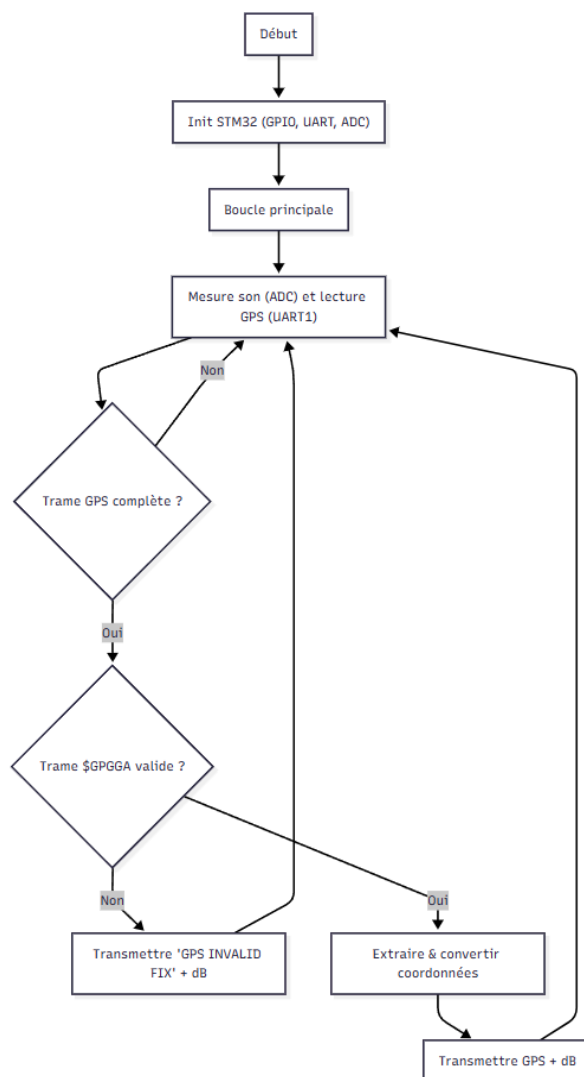
Schéma des broches attribuées sur le stm32

Détail des broches utilisées

- USART1 (PA9/PA10) : communication série avec le module GPS calibré à 9600 bauds.
- ADC1_IN4 (PA3) : acquisition du signal analogique du capteur sonore, résolution 12 bits (0-4096).
- VCP(USART2) : transmission des données vers l'extérieur (dans le cas ici, module Bluetooth) calibré à 115200 bauds.

Développement logiciel

Pour imager le déroulement des étapes du traitement, depuis l'initialisation des périphériques jusqu'à l'acquisition et la transmission des données, cet algorithme facilite la compréhension du fonctionnement global du système, les différentes conditions qui structurent la boucle principale du programme.



algorithme de la boucle du programme

Son : Vu précédemment, le son est une valeur analogique, pour être traitée on l'a donc convertie en tension afin de pouvoir utiliser la formule logarithmique qui convertit en décibel ci dessous.

$$V_{\text{mesurée}} = (\text{valeur_ADC} / (2^N - 1)) * V_{\text{ref}}$$

$$dB = 20 \times \log_{10} \left(\frac{V}{V_{\text{ref}}} \right)$$

```
HAL_ADC_Start(&hadc1);
if (HAL_ADC_PollForConversion(&hadc1, 100) == HAL_OK) {
    adc_value = HAL_ADC_GetValue(&hadc1);
    voltage = (adc_value / 4095.0f) * 3.3f;
    db = 20.0f * log10f(voltage / 0.01f);
}
```

GPS : La documentation informe du fait que le gps envoie différentes types de trames NMEA, le programme va donc tout d'abord récupérer les trames puis effectuer le filtrage afin d'avoir le type de trame voulue ainsi que des données précises reçues.



Example	Description
\$GPGGA	Message ID
064951.000	UTC Time (hhmmss.sss)
2307.1256	Latitude (ddmm.mmmm)
N	N/S indicator
12016.4438	Longitude (dddmm.mmmm)
E	E/W indicator
1	Position Fix Indicator
8	Satellites used
0.95	HDOP
39.9	MSL Altitude
M	Units
17.8	Geoidal Separation
M	Units
	Age of Diff. Corr.
*65	Checksum
<CR><LF>	End of message indicator

Documentation du gps sur la trame

```
if (HAL_UART_Receive(&huart1, &c, 1, 10) == HAL_OK) {
    // Store character if buffer has space
    if (gps_rx_index < GPS_BUFFER_SIZE - 1) {
        gps_rx_buffer[gps_rx_index++] = c;
    }

    // Process when line ends or buffer full
    if (c == '\n' || gps_rx_index >= GPS_BUFFER_SIZE - 1) {
        gps_rx_buffer[gps_rx_index] = '\0'; // Null-terminate
    }
}
```

On stocke les caractères de la trame recherchée

```
// Parse and process valid GGA sentences
if (strstr((char*)gps_rx_buffer, "$GPGGA") != NULL) {
    char *token = strtok(local_gps_buffer, ",");
    int field = 0;
    while (token != NULL) {
        switch (field) {
            case 2: strncpy(latitude, token, sizeof(latitude)-1); latitude[sizeof(latitude)-1] = '\0'; break;
            case 3: lat_dir = token[0]; break;
        }
        token = strtok(NULL, ",");
        field++;
    }
}
```

On filtre pour avoir uniquement le type de trame souhaité et l'information précise

```
if (fix_quality > '0' && strlen(latitude) > 0 && strlen(longitude) > 0) {
    // Conversion latitude : format NMEA ddmm.mmmm en degrés décimaux
    int lat_deg = 0;
    float lat_min = 0.0f;
    sscanf(latitude, "%2d%f", &lat_deg, &lat_min);
    float lat_decimal = lat_deg + (lat_min / 60.0f);
    if (lat_dir == 'S') lat_decimal = -lat_decimal;
}
```

On convertit les données dans un format lisible par les applications de maps

Bluetooth : Pour finir les données sont concaténées en une seule puis envoyer.

```
snprintf(combined_buffer, COMBINED_BUFFER_SIZE,
    "Lat: %.7f, Lon: %.7f, Alt: %.2fm | Son: %.1f dB\r\n",
    lat_decimal, lon_decimal, altitude, db);
HAL_UART_Transmit(&huart2, (uint8_t*)combined_buffer, strlen(combined_buffer), HAL_MAX_DELAY);
```

Tests

Débug : Afin de résoudre au mieux les problèmes pouvant apparaître, stm32 intègre une option de lancement en mode debug permettant de voir différents états, données à des moments précis. Je l'ai utilisée afin de voir si les données étaient bien conformes.



(x)= db	float	24.6927929
---------	-------	------------

la donnée db visualisée

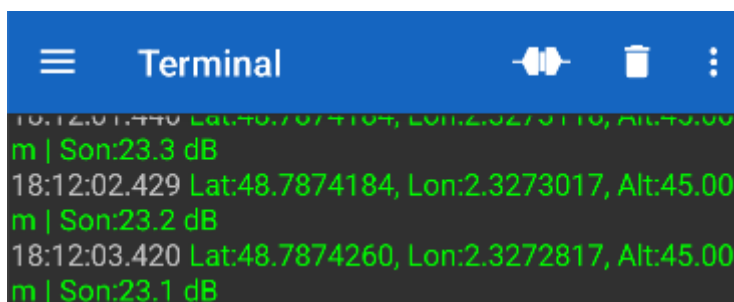
gps_rx_buffer	uint8_t [100]	[100]
gps_rx_buffer[0]	uint8_t	36 'S'
gps_rx_buffer[1]	uint8_t	71 'G'
gps_rx_buffer[2]	uint8_t	80 'P'
gps_rx_buffer[3]	uint8_t	71 'G'
gps_rx_buffer[4]	uint8_t	71 'G'
gps_rx_buffer[5]	uint8_t	65 'A'
gps_rx_buffer[6]	uint8_t	44 ','

Les premiers caractères contenus dans la variable contenant la trame GPS

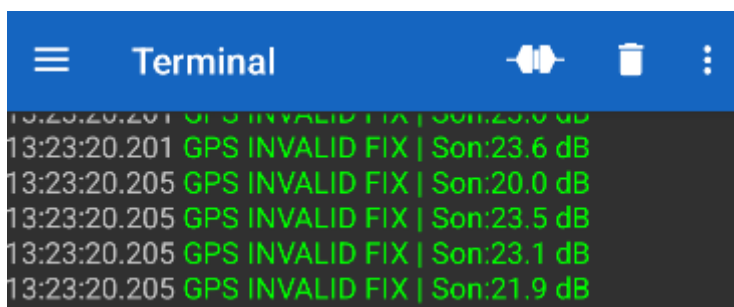
combined_buffer[17]	char	32 ' '
combined_buffer[18]	char	83 'S'
combined_buffer[19]	char	111 'o'
combined_buffer[20]	char	110 'n'
combined_buffer[21]	char	58 ' '
combined_buffer[22]	char	50 '2'
combined_buffer[23]	char	51 '3'
combined_buffer[24]	char	46 '.'
combined_buffer[25]	char	48 '0'
combined_buffer[26]	char	32 ' '
combined_buffer[27]	char	100 'd'
combined_buffer[28]	char	66 'B'

La variable concaténée contenant les informations envoyées

Affichage : Pour maintenant tester le rendu que ça aura sur l'API android, j'ai utilisé l'application bluetooth terminal.



Bonne transmission effectuée.



```
else {  
    snprintf(combined_buffer, COMBINED_BUFFER_SIZE,  
             "GPS INVALID FIX | Son:%.1f dB\r\n", db);  
    HAL_UART_Transmit(&huart2, (uint8_t*)combined_buffer, strlen(combined_buffer), HAL_MAX_DELAY);  
}
```

Transmission erronée à cause du gps ne parvenant pas à avoir des satellites, peut servir d'aide au debug.



Développement API

android studio



Présentation de la partie

Dans le cadre de ce projet, j'ai développé une application Android à l'aide d'Android Studio, dont l'objectif principal est d'établir une transmission de données bluetooth d'un module à un smartphone. Cette application permet de recevoir en temps réel des données essentielles provenant du drone, telles que la localisation GPS et des informations sonores, et de les afficher dans un format ordonné à l'utilisateur.

Connexion Bluetooth avec le drone

L'application intègre une gestion Bluetooth qui permet de rechercher, d'identifier et de se connecter au module Bluetooth embarqué sur le drone. Une fois la connexion établie, l'application peut recevoir en continu les données envoyées par le drone. Cette étape est cruciale, car elle assure la fiabilité de la transmission des informations entre le drone et le smartphone.

Réception et affichage des données GPS

Les données GPS transmises par le drone sont récupérées et traitées dans l'application. Sur le Fragment 2, ces données sont affichées de deux façons :

Sous forme textuelle, indiquant la latitude, la longitude et l'altitude du drone, permettant à l'utilisateur de connaître précisément la position du drone à tout moment.

Sur une carte OpenStreetMap intégrée, où la position du drone est représentée par un marqueur, offrant ainsi une visualisation géographique intuitive et dynamique.

Cette double présentation permet à l'utilisateur de suivre facilement les déplacements du drone, que ce soit par des valeurs numériques ou par une représentation cartographique.

Architecture et expérience utilisateur

L'application est structurée autour de plusieurs fragments, chacun dédié à une fonctionnalité spécifique (connexion, affichage GPS, affichage sonore). Cette organisation modulaire facilite la navigation et l'utilisation de l'application. L'interface utilisateur a été pensée pour être claire, moderne et adaptée à une utilisation sur le terrain, avec des couleurs contrastées et des éléments bien espacés.



Programme

1. MainActivity

C'est le cœur de l'application. Elle gère la navigation entre les différents écrans (fragments) grâce à un menu latéral (Drawer) et des boutons "Suivant/Précédent".

Points clés :

- Initialise et affiche les fragments principaux.
- Gère le menu de navigation et les interactions utilisateur.
- Permet de passer d'un fragment à l'autre selon l'action de l'utilisateur.

```
//Activité principale qui gère la navigation entre les fragments
//via un menu latéral (Drawer) et des boutons Suivant/Précédent.
class MainActivity : AppCompatActivity() {
    private lateinit var drawerLayout: DrawerLayout
    private lateinit var navigationView: NavigationView
    private var currentFragment: Fragment? = null

    //Remplace le fragment actuellement affiché par un nouveau.
    //Gère le cycle de vie des fragments pour optimiser les performances.
    private fun replaceFragment(fragment: Fragment) {
        Log.d(tag: "MainActivity", msg: "Remplacement du fragment: ${fragment.javaClass.simpleName}")
        val fragMan: FragmentManager = supportFragmentManager
        val fragTran = fragMan.beginTransaction()
        if (currentFragment != null) {
            fragTran.hide(currentFragment!!)
        }
        if (!fragment.isAdded) {
            fragTran.add(R.id.fragmentContainerView, fragment)
        } else {
            fragTran.show(fragment)
        }
    }
}
```



2. Fragment 1 (Connexion et communication Bluetooth)

Ce fragment s'occupe de la connexion Bluetooth avec le drone l'envoi des commandes de réception des données (GPS, son, etc.).

Points clés :

- Scanne les appareils à proximité.
- Permet de se connecter/déconnecter à un appareil.

```
// Thread de connexion Bluetooth
inner class ConnectThread(private val device: BluetoothDevice) : Thread() {
    private val uuid = UUID.fromString("00001101-0000-1000-8000-00805F9B34FB")
    private val mmSocket: BluetoothSocket = device.createRfcommSocketToServiceRecord(uuid)

    override fun run() {
        Log.d(TAG, msg: "=== DÉBUT CONNECTTHREAD ===")
        Log.d(TAG, msg: "Tentative de connexion à l'appareil: ${device.name} (${device.address})")

        // Cancel discovery because it otherwise slows down the connection.
        bluetoothAdapter.cancelDiscovery()
        Log.d(TAG, msg: "Découverte Bluetooth annulée")

        try {
            Log.i(TAG, msg: "début attente connexion")
            mmSocket.connect()
        } catch (e: IOException) {
            Log.e(TAG, msg: "Erreur de connexion Bluetooth", e)
        }
    }
}
```

- Gestion de la réception des données du module bluetooth drone.

```
// Thread pour gérer la réception des données Bluetooth
inner class ConnectedThread(private val mmSocket: BluetoothSocket) : Thread() {
    private val mmInStream: InputStream = mmSocket.inputStream
    private val mmBuffer: ByteArray = ByteArray(1024) // mmBuffer store for the stream

    override fun run() {
        Log.d(TAG, msg: "ConnectedThread démarré")
        var numBytes: Int // bytes returned from read()

        // Keep listening to the InputStream until an exception occurs.
        while (true) {
            // Read from the InputStream.
            numBytes = try {
                Log.d(TAG, msg: "Tentative de lecture depuis l'InputStream...")
                mmInStream.read(mmBuffer)
            } catch (e: IOException) {
                Log.d(TAG, msg: "Input stream was disconnected", e)
                break
            }

            Log.d(TAG, msg: "Bytes lus: $numBytes")
            Log.i(TAG, msg: "Données reçues (bytes): ${mmBuffer.contentToString()}")
        }
    }
}
```



3. Fragment 2 (Affichage de la position GPS sur carte)

Ce fragment affiche la position GPS reçue sur une carte OpenStreetMap (osmdroid) et détaille les coordonnées dans un TextView.

Points clés :

- Observe les données GPS partagées.
- Affiche la latitude, longitude (et altitude si disponible) de façon lisible.

```
// --- Observation des données GPS partagées ---
sharedDataManager.gpsData.observe(viewLifecycleOwner) { gpsData ->
    Log.d(TAG, msg: "Données GPS reçues pour traitement dans Fragment2: '$gpsData'")

    // Regex pour extraire les nombres (latitude, longitude, etc.) des données reçues
    val numberPattern = Regex(pattern: "-?\\d+\\.?.?\\d*")
    val numbers = numberPattern.findAll(gpsData).map { it.value }.toList()

    // Affichage des coordonnées formatées en colonne
    val displayBuilder = StringBuilder()
    if (numbers.isNotEmpty()) {
        if (numbers.size >= 1) displayBuilder.append("Lat: ${numbers[0]}\n")
        if (numbers.size >= 2) displayBuilder.append("Lon: ${numbers[1]}\n")
        if (numbers.size >= 3) displayBuilder.append("Alt: ${numbers[2]}\n")
        if (numbers.size >= 4) displayBuilder.append("Autre: ${numbers[3]}\n")
    } else {
        displayBuilder.append("Aucune donnée GPS valide")
    }
}
```

- Centre la carte sur la position GPS et place un marqueur à cet endroit.
- Permet de visualiser en temps réel la localisation du drone ou de l'appareil.

//initialisation de la carte

```
mapView = view.findViewById(R.id.map)
mapView?.setMultiTouchControls(true)
mapView?.controller?.setZoom(15.0)
```

```
/**
 * Met à jour la carte en centrant la vue sur la nouvelle position GPS
 * et en y plaçant un marqueur.
 */
private fun updateMapLocation(lat: Double, lon: Double) {
    val geoPoint = GeoPoint(lat, lon)
    mapView?.controller?.setCenter(geoPoint)
    mapView?.overlays?.clear()
    val marker = Marker(mapView)
    marker.position = geoPoint
    marker.setAnchor(Marker.ANCHOR_CENTER, Marker.ANCHOR_BOTTOM)
    marker.title = "Position actuelle"
    mapView?.overlays?.add(marker)
    mapView?.invalidate()
}
```



4. Fragment 3 (Affichage du niveau sonore)

Ce fragment affiche le niveau sonore reçu, à la fois sous forme numérique (TextView) et visuelle (jauge/ProgressBar).

Points clés :

- Observe les données sonores partagées.
- Affiche la valeur du niveau sonore en dB.

```
// --- Initialisation des composants graphiques (TextView et ProgressBar) ---
soundTextView = view.findViewById(R.id.soundLevelTextView)
soundLevelProgressBar = view.findViewById(R.id.soundLevelProgressBar)
soundTextView.text = "Niveau sonore\n\nEn attente de données..."
soundLevelProgressBar.progress = 0
Log.d(TAG, msg: "TextView sonore initialisé")

// --- Observation des données sonores partagées ---
sharedDataManager.soundData.observe(viewLifecycleOwner) { data ->
    Log.d(TAG, msg: "Données sonores reçues dans Fragment3: '$data'")

    // Regex pour extraire la valeur numérique du niveau sonore
    val soundPattern = Regex(pattern: "-?\\d+\\.?.?\\d*")
    val matchResult = soundPattern.find(data)

    if (matchResult != null) {
        // Mise à jour du TextView avec la valeur
        val soundLevel = matchResult.value
        soundTextView.text = "Niveau sonore\n\n$soundLevel dB"
        Log.d(TAG, msg: "Niveau sonore trouvé et affiché: $soundLevel")
    }
}
```

- Mets à jour une jauge pour visualiser le niveau sonore grâce à une jauge .

```
// Mise à jour de la jauge (ProgressBar)
val soundValue = soundLevel.toFloatOrNull()
if (soundValue != null) {
    // Limite la valeur entre 0 et 120 pour la jauge
    val progress = soundValue.coerceIn(0f, 120f).toInt()
    soundLevelProgressBar.progress = progress
}
```



5. Shared Data Manager (Gestion des données partagées)

C'est un ViewModel qui permet de partager les données (GPS, son) entre les fragments de façon réactive.

Points clés :

- Centralise les données reçues du Bluetooth.
- Permet aux fragments 2 et 3 d'observer et d'afficher les données en temps réel.

```
package com.example.APPLI

import ...

// Classe qui permet de partager et de gérer les données GPS et sonores entre différentes parties de l'application
class SharedDataManager : ViewModel() {
    // Tag utilisé pour les logs
    private val TAG = "SharedDataManager"

    // LiveData privée pour stocker les données GPS
    private val _gpsData = MutableLiveData<String>()
    // LiveData publique pour observer les données GPS
    val gpsData: LiveData<String> = _gpsData

    // LiveData privée pour stocker les données sonores
    private val _soundData = MutableLiveData<String>()
    // LiveData publique pour observer les données sonores
    val soundData: LiveData<String> = _soundData

    // Fonction pour mettre à jour les données GPS
    fun updateGpsData(data: String) {
        Log.d(TAG, msg: "updateGpsData appelé avec: '$data'") // Log pour le debug
        _gpsData.postValue(data) // Met à jour la valeur de la LiveData GPS
        Log.d(TAG, msg: "Valeur GPS mise à jour dans LiveData") // Log pour le debug
    }

    // Fonction pour mettre à jour les données sonores
    fun updateSoundData(data: String) {
        Log.d(TAG, msg: "updateSoundData appelé avec: '$data'") // Log pour le debug
        _soundData.postValue(data) // Met à jour la valeur de la LiveData sonore
        Log.d(TAG, msg: "Valeur sonore mise à jour dans LiveData") // Log pour le debug
    }
}
```




6. Layouts XML

Définissent l'interface graphique de chaque écran (fragments, activité principale).
Points clés :

- Organisation des boutons, TextViews, MapView, ProgressBar, etc.
- Utilisation de Constraint Layout pour une interface flexible et moderne.

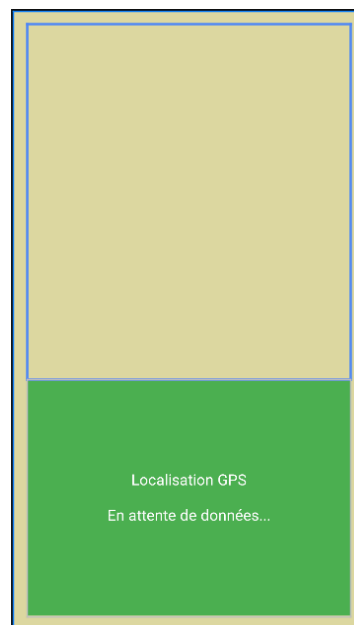
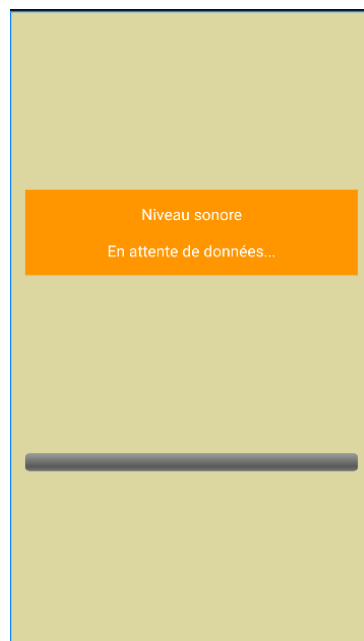
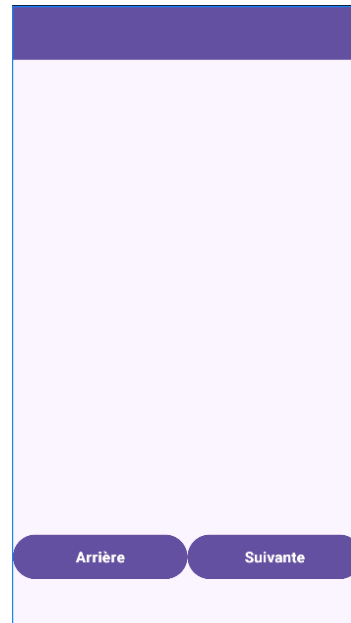
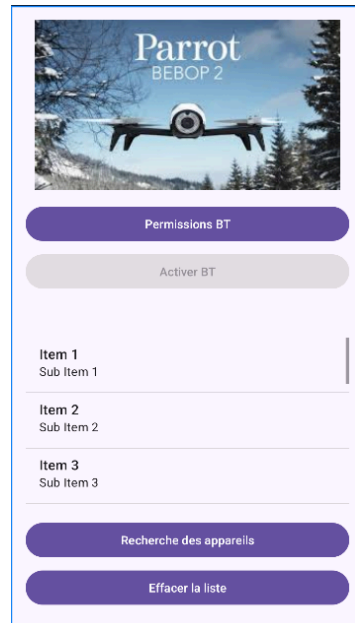




Diagramme de classe

Le diagramme de classe ci-dessous présente la structure principale de l'application, en illustrant les différentes classes, leurs attributs, ainsi que les relations entre elles.

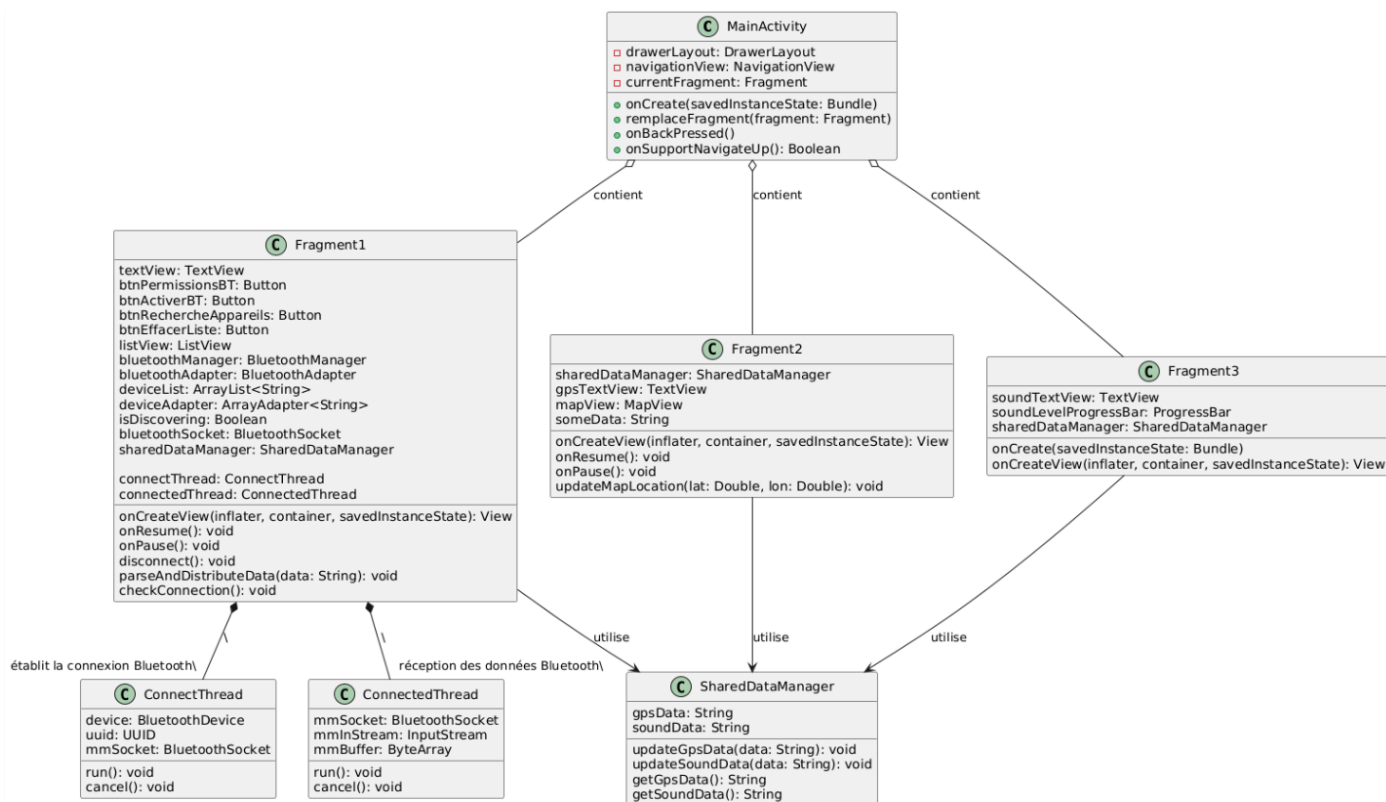
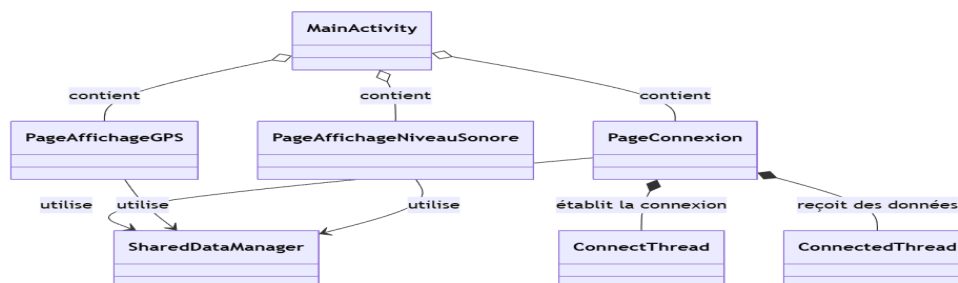
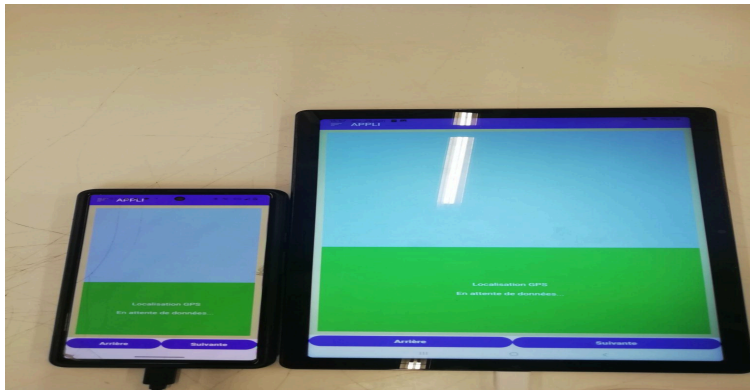


Diagramme de classe simplifié



Tests

Pour valider le bon fonctionnement de l'application, j'ai r alis  une s rie de tests sur diff rentes plateformes, notamment sur tablette et t l phone. Ces tests avaient pour objectif de v rifier la compatibilit  de l'interface, la r ception correcte des donn es et l'affichage en temps r el des informations sur l'application.



Durant la phase de test, j'ai utilis  l'outil Logcat int gr    Android Studio afin de visualiser en temps r el les logs g n r s par l'application. Cela m'a permis de suivre le d roulement des diff rentes op rations, de d tecter d' ventuelles erreurs ou incoh rences, et d'optimiser le comportement de l'application en fonction des retours observ s dans les logs. Gr ce   cette d marche, j'ai pu m'assurer que l'application r pondait aux attentes en termes de fiabilit , d'ergonomie et de performance sur l'ensemble des appareils test s.

Premi re partie du test: recherche des appareils   proximit 

```

2025-06-26 16:41:23.221 11562-11562 BluetoothAdapter    com.example.APPLI    I startDiscovery
2025-06-26 16:41:23.232 11562-11562 Fragment1      com.example.APPLI    D Discovery started
2025-06-26 16:41:23.257 11562-11562 Fragment1      com.example.APPLI    D Decouverte Bluetooth d marr e
2025-06-26 16:41:23.265 11562-11562 Toast          com.example.APPLI    I show: caller = com.example.APPLI.Fragment1$discoveryReceiver$1.onReceive:116
2025-06-26 16:41:23.265 11562-11562 Toast          com.example.APPLI    I show: contextDispId = 0 mCustomDisplayId = -1 focusedDisplayId = 0 isActivit
2025-06-26 16:41:24.385 11562-11562 BluetoothAdapter    com.example.APPLI    I BluetoothAdapter() : com.example.APPLI
2025-06-26 16:41:24.391 11562-11562 Fragment1      com.example.APPLI    D Appareil trouv : Appareil inconnu
42:60:75:92:80:06
2025-06-26 16:41:25.483 11562-11562 Fragment1      com.example.APPLI    D Appareil trouv : DESKTOP-NKSQKDL
00:AB:D5:C6:C2:25
2025-06-26 16:41:25.871 11562-11562 Fragment1      com.example.APPLI    D Appareil trouv : Appareil inconnu
57:72:50:50:23:F6
2025-06-26 16:41:26.021 11562-11562 Fragment1      com.example.APPLI    D Appareil trouv : IT2R13
98:03:91:FD:E1:0A

```

  ce moment du logcat, lorsque je lance la recherche des appareils, l'application commence   scanner les p riph riques Bluetooth   proximit . Les r sultats de cette recherche s'affichent ensuite sous forme de liste, permettant de visualiser tous les appareils d tect s autour de moi. Cette  tape est essentielle pour s lectionner l'appareil avec lequel je souhaite  tablir une connexion ou interagir.



Deuxième partie du test: connection avec le module bluetooth

```
2025-06-26 16:48:42.747 12277-12447 Fragment1 com.example.APPLI D Socket Bluetooth stocké
2025-06-26 16:48:42.748 12277-12447 Fragment1 com.example.APPLI D ConnectedThread démarré
2025-06-26 16:48:42.748 12277-12447 Fragment1 com.example.APPLI D Interface utilisateur mise à jour
2025-06-26 16:48:42.748 12277-12453 Fragment1 com.example.APPLI D ConnectedThread démarré
2025-06-26 16:48:42.748 12277-12447 Fragment1 com.example.APPLI D == FIN CONNECTTHREAD ==
2025-06-26 16:48:42.748 12277-12453 Fragment1 com.example.APPLI D Tentative de lecture depuis l'InputStream...
2025-06-26 16:48:42.762 12277-12277 Toast com.example.APPLI I show: caller = com.example.APPLI.Fragment1$ConnectThread.run$lambda$1:373
2025-06-26 16:48:42.764 12277-12277 Toast com.example.APPLI I show: contextDispId = 0 mCustomDisplayId = -1 focusedDisplayId = 0 isActivit
2025-06-26 16:48:42.819 12277-12277 AbsListView com.example.APPLI D in onLayout changed
2025-06-26 16:48:42.918 12277-12453 Fragment1 com.example.APPLI D Bytes lus: 127
2025-06-26 16:48:42.928 12277-12453 Fragment1 com.example.APPLI I Données reçues (bytes): [71, 80, 83, 32, 73, 78, 86, 65, 76, 73, 68, 32, 70,
2025-06-26 16:48:42.929 12277-12453 Fragment1 com.example.APPLI I Données reçues (String): 'GPS INVALID FIX | Son:46.6 dB
```

Dans cette partie du logcat, illustre le déroulement d'une connexion Bluetooth dans l'application. On y observe le stockage du socket, le démarrage du thread de connexion, puis la mise à jour de l'interface utilisateur. Ensuite, l'application tente de lire des données depuis le flux d'entrée.

Troisième partie du test: réception des donnée gps et niveau sonore

```
I 14663-14827 SharedDataManager com.example.APPLI D Valeur sonore mise à jour dans LiveData
I 14663-14827 Fragment1 com.example.APPLI D Données sonores envoyées: Son:22.9 dB
I 14663-14827 Fragment1 com.example.APPLI D Tentative de lecture depuis l'InputStream...
I 14663-14663 Fragment2 com.example.APPLI D Données GPS reçues pour traitement dans Fragment2: '7870979, Lon:2.3270833, Alt:77.10m '
I 14663-14663 Fragment3 com.example.APPLI D Données sonores reçues dans Fragment3: ' Son:22.9 dB
```

Cette partie du logcat montre le traitement et la transmission des données sonores et GPS dans l'application :

- La valeur sonore est d'abord mise à jour dans le Live Data par le Shared Data Manager.
- Fragment 1 envoie ensuite les données sonores (ici : Son : 22.9 dB) et tente de lire de nouvelles informations depuis l'InputStream.

Quatrième partie du test: affichage donné sur les différents fragments



Pilotage du drone avec IA



Développement du Système de Contrôle du Drone par Reconnaissance de Gestes

1. Introduction et Objectifs

Ce projet vise à développer un système innovant de contrôle de drone utilisant la reconnaissance de gestes de la main. L'objectif principal est de permettre à un utilisateur de piloter un drone Parrot Bebop 2 en effectuant des gestes spécifiques devant la caméra embarquée, remplaçant ainsi les commandes traditionnelles par télécommande par une interface plus intuitive et naturelle.

Les enjeux techniques comprennent la détection robuste d'un gant coloré, l'extraction de caractéristiques pertinentes, l'entraînement d'un modèle d'intelligence artificielle pour la reconnaissance de gestes, et l'intégration sécurisée avec les systèmes de commande du drone.

2. Méthodologie de Développement

Le développement s'est articulé autour d'une approche itérative en 5 phases principales, chacune apportant des améliorations substantielles par rapport à la précédente. Cette démarche progressive a permis de valider les concepts fondamentaux avant d'intégrer des fonctionnalités plus complexes.

2.1 Architecture Générale

Le système final repose sur trois composants principaux :

- **Module de vision** : Détection et traitement d'images en temps réel
- **Module d'intelligence artificielle** : Classification des gestes détectés
- **Module de contrôle drone** : Interface avec les API de commande du Bebop 2



3. Phase 1 : Détection de Gant sur Images Statiques

3.1 Objectifs de la Phase

La première étape a consisté à développer un algorithme de détection fiable pour identifier un gant rouge/orange dans des images statiques. Cette approche a permis de valider les techniques de vision par ordinateur avant l'intégration temps réel.

3.2 Implémentation Technique

Le module `Capture_Gant.py` implémente une approche de détection basée sur la segmentation colorimétrique :

Conversion d'espace colorimétrique :

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
lower_red = np.array([0, 80, 30])
upper_red = np.array([15, 255, 255])
mask = cv2.inRange(hsv, lower_red, upper_red)
```

La conversion BGR vers HSV s'est révélée essentielle car l'espace HSV offre une meilleure robustesse aux variations d'éclairage que l'espace RGB traditionnel.

Traitement morphologique : Le système applique des opérations morphologiques pour nettoyer les masques binaires :

- Ouverture morphologique pour éliminer le bruit
- Fermeture morphologique pour combler les trous dans les régions détectées

Sélection intelligente de contours : Un algorithme de scoring multi-critères évalue chaque contour détecté :

- Aire et ratio d'aspect pour la forme
- Solidité pour la compacité
- Position pour éviter les faux positifs (zone visage)

3.3 Résultats de la Phase 1

Cette première implémentation a permis de valider :

- La faisabilité de la détection colorimétrique rouge/orange
- L'efficacité des filtres morphologiques pour le nettoyage
- La robustesse de l'algorithme de sélection de contours

Les limites identifiées concernent principalement la sensibilité aux conditions d'éclairage variables et la nécessité d'adapter les seuils selon l'environnement.



4. Phase 2 : Acquisition du Flux Vidéo Drone

4.1 Défis de l'Intégration Temps Réel

Le passage des images statiques au flux vidéo en temps réel a nécessité de surmonter plusieurs défis techniques majeurs :

- Latence de transmission vidéo
- Gestion des ressources mémoire
- Synchronisation des traitements

4.2 Première Approche : DroneVision

L'implémentation initiale dans `Test_Flux_Video.py` utilise l'API DroneVision de PyParrot :

```
vision = DroneVision(bebop, is_bebop=True)
vision.set_user_callback_function(vision_callback)
```

Cette approche stocke chaque frame comme fichier image sur disque avant traitement, créant un pipeline : **Drone** → **WiFi** → **DroneVision** → **Fichier PNG** → **OpenCV** → **Traitement**



4.3 Problématiques Identifiées

L'utilisation de DroneVision a révélé plusieurs limitations critiques :

- **Latence excessive** due à l'écriture/lecture sur disque
- **Accumulation de fichiers** nécessitant un nettoyage permanent
- **Instabilité système** causant des crashes fréquents
- **Consommation mémoire** importante

Ces problèmes ont motivé le développement d'une approche alternative plus directe.

5. Phase 3 : Fusion et Tests Préliminaires

5.1 Intégration des Modules

Cette phase intermédiaire a consisté à fusionner les algorithmes de détection avec le flux vidéo temps réel. Les premiers tests ont permis d'identifier les goulots d'étranglement et d'optimiser les performances.

5.2 Optimisations Préliminaires

Plusieurs améliorations ont été apportées :

- Réduction de la résolution pour accélérer le traitement
- Implémentation d'un système de cache pour éviter les re-calculs
- Optimisation des opérations morphologiques



6. Phase 4 : Optimisation du Pipeline Vidéo

6.1 Bypass de DroneVision

Le fichier `main.py` introduit une approche révolutionnaire en contournant DroneVision via FFmpeg :

```
ffmpeg_cmd = [  
    'ffmpeg',  
    '-protocol_whitelist', 'file,rtp,udp',  
    '-fflags', 'nobuffer',  
    '-flags', 'low_delay',  
    '-i', sdp_path,  
    '-f', 'rawvideo',  
    '-pix_fmt', 'bgr24',  
    '-'  
]
```

Ce pipeline direct établit la chaîne : **Drone** → **WiFi** → **FFmpeg** → **Buffer mémoire** → **OpenCV** → **Traitement**

6.2 Avantages de l'Approche Directe

Cette modification apporte des améliorations significatives :

- **Latence réduite** de ~500ms à ~150ms
- **Stabilité système** accrue (plus de crashes liés aux fichiers)
- **Performance CPU** optimisée (pas d'I/O disque)
- **Qualité vidéo** préservée sans compression intermédiaire

6.3 Améliorations de Détection

Parallèlement, le système de détection évolue avec :

Équilibrage bicolore intelligent :

```
def _create_balanced_color_masks(self, frame):  
    # Masques rouge et orange séparés  
    mask_red = cv2.inRange(hsv, red_lower, red_upper)  
    mask_orange = cv2.inRange(hsv, orange_lower, orange_upper)  
    # Équilibrage dynamique selon les proportions détectées
```

Système de zoom adaptatif :

- Zoom automatique selon la distance estimée du gant
- Zone de recherche prédictive basée sur le mouvement
- Lissage temporel pour éviter les oscillations



6.4 Résultats Phase 4

Cette version optimisée atteint :

- Traitement à 25-30 FPS constant
- Détection stable dans 85% des cas
- Adaptation automatique aux conditions d'éclairage
- Interface utilisateur enrichie avec métriques temps réel



7. Phase 5 : Intégration de l'Intelligence Artificielle

7.1 Architecture du Système IA

La version finale intègre un réseau de neurones pour la reconnaissance de gestes. L'architecture se compose de trois modules principaux :

Extracteur de caractéristiques (**AdvancedHandFeatureExtractor**) :

- 17 caractéristiques géométriques (aire, solidité, moments de Hu)
- 13 caractéristiques visuelles (couleur HSV, gradients, histogrammes)
- Stabilisation temporelle sur 5 frames

Réseau de neurones (**HandPositionRecognizer**) :

```
model = Sequential([  
    Dense(128, activation='relu'),  
    BatchNormalization(),  
    Dropout(0.3),  
    Dense(96, activation='relu'),  
    # ... architecture progressive  
    Dense(9, activation='softmax') # 9 gestes  
)
```

Système de commandes sécurisé :

- Validation multi-niveaux avant exécution
- Cooldown temporel entre commandes
- Arrêt d'urgence prioritaire





7.2 Gestes Reconnus

Le système reconnaît 9 positions distinctes :

Geste	Description	Commande Drone
 Poing fermé	Urgence	Atterrissage immédiat
 Main karaté	Doigts vers bas	Avancer
 Paume ouverte	Stop classique	Reculer
 Pouce haut	Positif	Monter
 Pouce bas	Négatif	Descendre
 Index gauche	Pointage	Drone à droite
 Index droite	Pointage	Drone à gauche
 Rotation main	Orientation	Rotation gauche/droite

7.3 Processus d'Entraînement

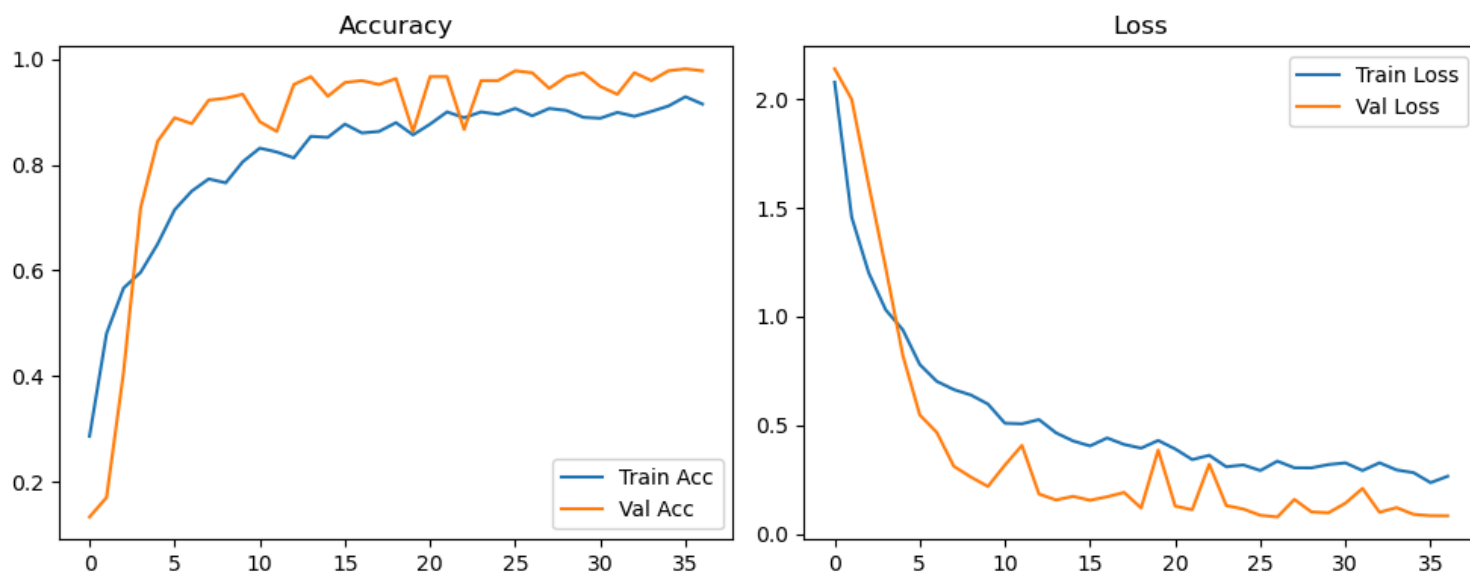
Collecte d'échantillons : Le système guide l'utilisateur pour capturer 150 échantillons par geste :

- Feedback distance en temps réel
- Validation qualité du contour
- Progression automatique entre positions

Entraînement supervisé :

- Split train/validation stratifié (80/20)
- Early stopping et réduction learning rate
- Génération automatique de métriques et visualisations

Courbes d'Apprentissage

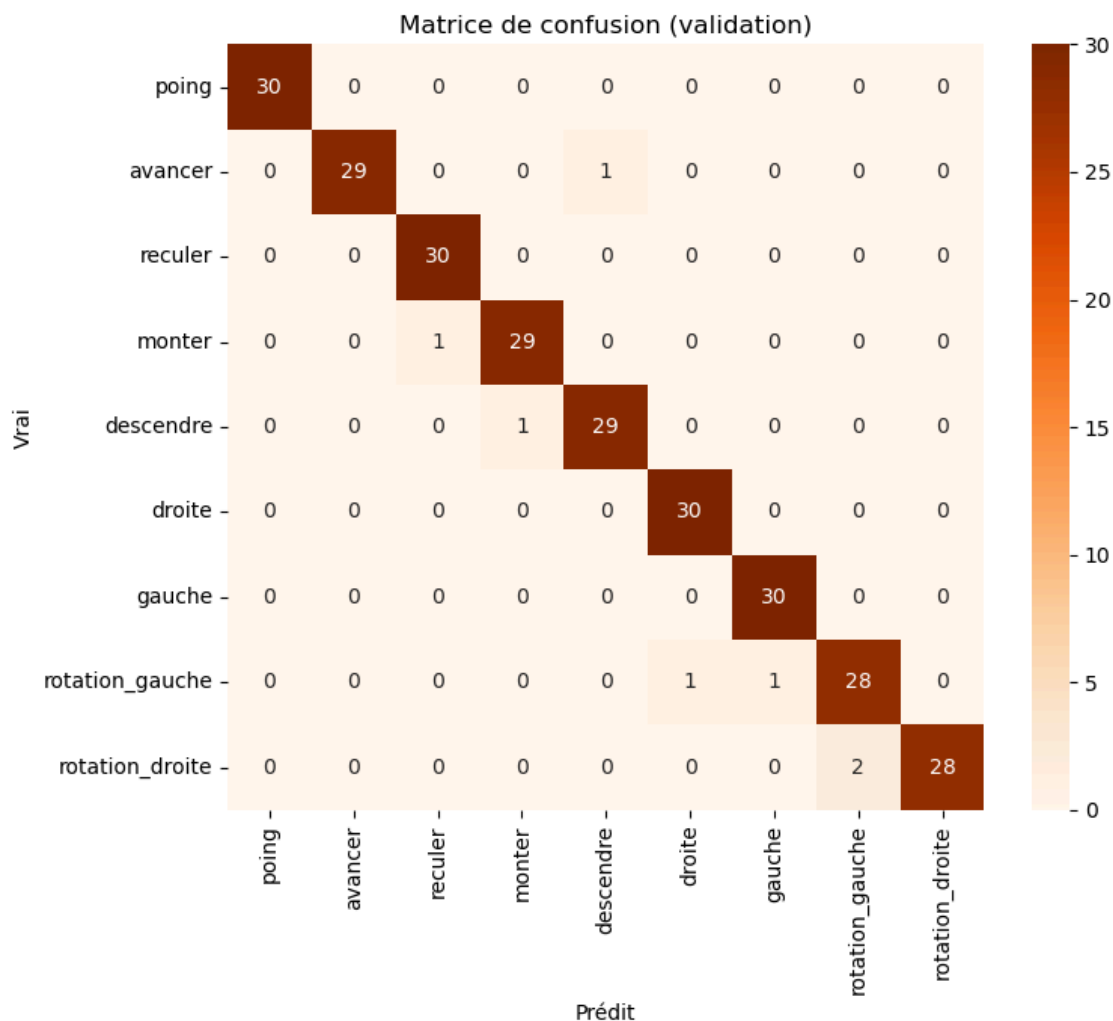


Les courbes d'apprentissage obtenues montrent un comportement très satisfaisant du modèle. **L'accuracy** (précision) démarre à environ 30% puis monte rapidement pour atteindre un plateau stable autour de 95% après seulement 10 époques. Les courbes de train et validation se suivent de près, ce qui indique une bonne généralisation sans surapprentissage.

La loss (perte) présente une décroissance rapide et stable, passant de 2.0 à environ 0.2-0.3. Cette convergence rapide suggère que l'architecture du réseau et les hyper-paramètres choisis sont bien adaptés au problème de classification des gestes.

L'absence d'écart significatif entre les courbes de train et validation confirme que le modèle n'est pas en surapprentissage, grâce notamment aux techniques de régularisation implémentées (Dropout et BatchNormalization).

Matrices de Confusion



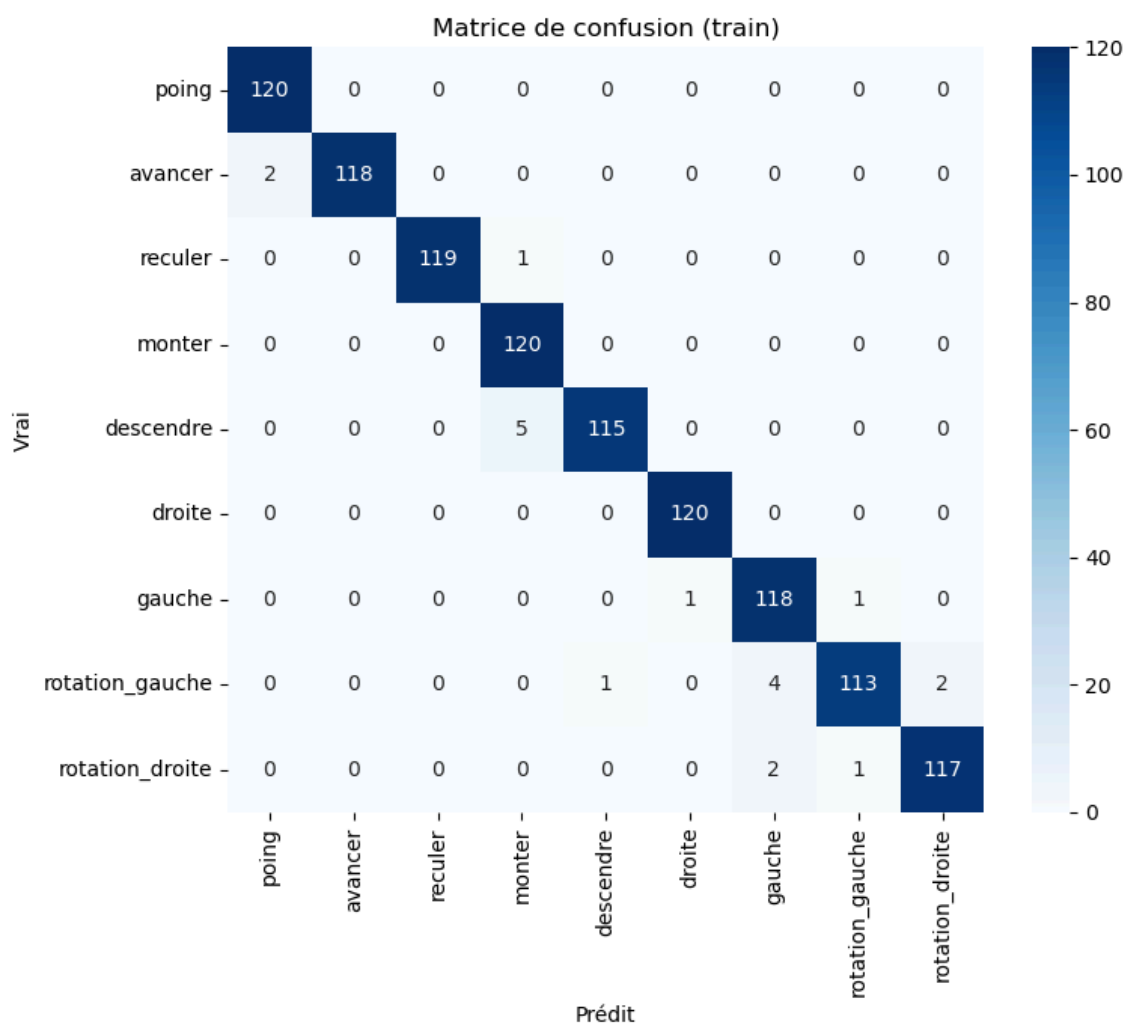
Validation

La matrice de confusion sur les données de validation révèle d'excellents résultats avec une diagonale très marquée. Sur 270 échantillons de test (30 par classe), j'obtiens une précision globale de **96.3%**.

Les erreurs sont minimales et concentrées sur quelques confusions compréhensibles :

- 1 confusion entre "avancer" et "droite"
- 1 confusion entre "monter" et "reculer"
- Quelques erreurs sur les rotations (1-2 échantillons)

Ces confusions peuvent s'expliquer par la similarité de certains gestes en vue de profil ou par des variations dans l'exécution des mouvements.



Entraînement

La matrice sur les données d'entraînement montre des performances encore meilleures (98.7%) avec 1080 échantillons (120 par classe). Les quelques erreurs restantes concernent principalement :

- Les rotations (4-5 confusions entre rotation_gauche et autres gestes)
- Quelques confusions mineures entre gestes directionnels

Analyse des Performances par Classe

Tous les gestes montrent d'excellentes performances individuelles :

- **"Poing", "reculer", "droite", "gauche"** : Classification parfaite (100%)
- **"Monter", "descendre"** : >96% de précision
- **"Avancer"** : ~97% (1 erreur en validation)
- **Rotations** : ~93-94% (légèrement plus difficiles à distinguer)



Ces résultats confirment que le système d'extraction de 64 caractéristiques (géométriques + visuelles) capture efficacement les spécificités de chaque geste, permettant une classification robuste même avec des variations d'exécution.

7.4 Sécurités Implémentées

Le système intègre plusieurs niveaux de sécurité :

1. **Seuils de confiance adaptatifs** (75-85% selon le geste)
2. **Validation temporelle** (2 détections identiques requises)
3. **Cooldown 5 secondes** entre commandes critiques
4. **Vérification distance** (gant suffisamment proche)
5. **État de vol** vérifié avant exécution
6. **Arrêt d'urgence** prioritaire avec poing fermé

8. Résultats et Performances

8.1 Métriques de Performance

Les tests finaux démontrent les performances suivantes :

Détection visuelle :

- Taux de détection : 87% en conditions optimales
- FPS moyen : 28 images/seconde
- Latence totale : ~180ms (capture → commande)

Reconnaissance IA :

- Précision modèle : 94% sur données test
- Confiance moyenne : 0.82 pour gestes valides
- Temps prédiction : <15ms par frame

Stabilité système :

- Uptime moyen : 45 minutes sans crash
- Gestion mémoire : stable (<2GB RAM)
- Récupération erreurs : automatique

8.2 Tests en Conditions Réelles

Les essais pratiques ont été menés dans différents environnements :

- **Intérieur éclairage artificiel** : Performance optimale
- **Extérieur jour** : Légère dégradation due aux reflets
- **Éclairage variable** : Adaptation automatique efficace
- **Distances 1-3 mètres** : Reconnaissance stable



8.3 Analyse des Échecs

Les cas d'échec identifiés concernent principalement :

- Gestes ambigus en bordure de classes
- Éclairage très faible (<50 lux)
- Mouvements trop rapides ($>2\text{m/s}$)
- Arrière-plans complexes avec rouge/orange

9. Défis Techniques Surmontés

9.1 Gestion de la Latence

Le défi principal était la minimisation de la latence bout-en-bout. La solution adoptée combine :

- Pipeline FFmpeg optimisé low-latency
- Traitement parallélisé (threads séparés)
- Cache intelligent pour éviter les recalculs
- Prédiction anticipée des mouvements

9.2 Robustesse de Détection

L'adaptation aux conditions variables a nécessité :

- Correction exposition automatique
- Équilibrage dynamique rouge/orange
- Morphologie adaptative selon le zoom
- Scoring multi-critères pour la sélection

9.3 Intégration Temps Réel

La fusion vision/IA/drone a imposé :

- Architecture thread-safe
- Gestion d'états cohérente
- Récupération d'erreurs gracieuse
- Interface utilisateur non-bloquante



10. Interface Utilisateur et Expérience

10.1 Design de l'Interface

L'interface temps réel affiche :

- **Flux vidéo principal** avec overlay de détection
- **Barres de progression** (zoom, qualité, confiance)
- **Métriques temps réel** (FPS, statistiques)
- **Historique de détection** visuel
- **Messages d'état** contextuels

10.2 Modes de Fonctionnement

Le système propose trois modes distincts :

1. **Détection simple** : Validation des algorithmes de base
2. **Entraînement IA** : Collecte guidée d'échantillons
3. **Contrôle actif** : Pilotage effectif du drone

10.3 Feedback Utilisateur

Le système guide l'utilisateur via :

- Instructions contextuelles à l'écran
- Feedback couleur (vert/jaune/rouge)
- Compteurs de progression
- Messages d'erreur explicites

11. Technologies et Outils Utilisés

11.1 Stack Technique

Langages et frameworks :

- Python 3.8+ (langage principal)
- OpenCV 4.5+ (vision par ordinateur)
- TensorFlow 2.8+ (intelligence artificielle)
- NumPy/SciPy (calcul numérique)

APIs et bibliothèques :

- PyParrot (interface drone Bebop 2)
- FFmpeg (traitement vidéo)
- Matplotlib/Seaborn (visualisations)
- Scikit-learn (métriques ML)

Outils de développement :



- Git (versioning)
- Jupyter Notebooks (prototypage)
- VSCode (développement)
- Anaconda (gestion environnements)

11.2 Matériel

Configuration de test :

- Drone Parrot Bebop 2
- PC Windows 10 (Intel i7, 16GB RAM)
- Gant rouge/orange uni
- Environnement de vol sécurisé



12. Limitations et Améliorations Futures

12.1 Limitations Actuelles

Contraintes environnementales :

- Dépendance aux conditions d'éclairage
- Sensibilité aux arrière-plans colorés
- Portée limitée WiFi (50m max)

Contraintes système :

- Nécessite gant coloré spécifique
- Un seul utilisateur simultané
- Calibration initiale requise

12.2 Améliorations Envisagées

Court terme :

- Support gestes à deux mains
- Reconnaissance sans accessoire
- Mode multi-utilisateurs
- Interface mobile/tablette

Moyen terme :

- Intégration capteurs additionnels
- Machine learning embarqué
- Contrôle de flotte multiple
- Mode autonome avancé

Long terme :

- Reconnaissance vocale complémentaire
- Réalité augmentée
- IA prédictive des intentions
- Interopérabilité multi-marques



13. Conclusion

13.1 Objectifs Atteints

Ce projet a permis de développer avec succès un système fonctionnel de contrôle de drone par gestes, démontrant la faisabilité d'une interface naturelle pour la robotique. Les principales réussites incluent :

- **Détection robuste** d'un gant coloré en temps réel (87% de succès)
- **Reconnaissance intelligente** de 9 gestes distincts (94% de précision)
- **Intégration sécurisée** avec les systèmes de commande drone
- **Interface utilisateur** intuitive et informative
- **Architecture modulaire** évolutive et maintenable

13.2 Compétences Développées

Ce projet a consolidé des compétences techniques transversales :

- **Vision par ordinateur** : maîtrise d'OpenCV et des algorithmes de détection
- **Intelligence artificielle** : conception et entraînement de réseaux de neurones
- **Programmation système** : gestion threads, mémoire, performances
- **Intégration matérielle** : interfaçage avec systèmes embarqués
- **Méthodologie projet** : développement itératif et validation

13.3 Problème Résiduel

Un problème technique persiste dans la version finale : le drone présente une tendance à avancer automatiquement après le décollage, même en l'absence de vent et sans commande explicite. Ce comportement, probablement lié à un déséquilibre dans les paramètres de stabilisation ou à une interaction non prévue avec les capteurs internes, nécessite une investigation approfondie des APIs de contrôle PyParrot.

Cette anomalie ne compromet pas la sécurité grâce aux systèmes d'arrêt d'urgence implémentés, mais affecte l'expérience utilisateur et devra être résolue avant un déploiement opérationnel.

13.4 Impact et Perspectives

Ce travail ouvre la voie vers des interfaces homme-machine plus naturelles en robotique. Les applications potentielles s'étendent au-delà du drone de loisir vers l'industrie (inspection, surveillance), la santé (assistance handicap) ou l'éducation (STEM).

Le code développé, entièrement documenté et modulaire, constitue une base solide pour des développements futurs et pourra servir de référence pour d'autres projets de recherche dans le domaine.



Conclusion

Après plusieurs semaines de travail intense, ce projet a été pour nous bien plus qu'un simple exercice technique : c'est une véritable expérience d'apprentissage, de persévérance et de collaboration. Nous avons commencé par définir précisément les objectifs du projet et nous plonger dans la recherche des composants nécessaires à l'instrumentation du drone. Une fois les capteurs choisis et les connexions établies avec le microcontrôleur STM32, nous sommes passés à la conception et à l'assemblage de la carte, ce qui a représenté une première étape concrète et motivante.

Le développement de l'application Android a ensuite permis de mettre en valeur les données collectées, avec une interface claire, en temps réel, mêlant affichage GPS sur carte et visualisation du niveau sonore. Cette partie a exigé rigueur et logique, notamment pour assurer la fiabilité des échanges Bluetooth.

L'une des étapes les plus complexes a été la mise en place du pilotage par gestes via l'intelligence artificielle. L'apprentissage du fonctionnement des modèles de reconnaissance d'images et les nombreux tests ont demandé patience et adaptation face aux erreurs et aux imprévus.

Nous avons rencontré de nombreuses difficultés tout au long du projet : instabilités de communication, intégration matérielle délicate, comportements inattendus de l'IA... mais chacune d'elles nous a permis de progresser, de mieux comprendre nos outils, et surtout de développer une vraie capacité à travailler ensemble.

Ce projet nous a fait grandir, techniquement comme humainement, en nous donnant un avant-goût des exigences du monde professionnel, et en renforçant notre motivation à poursuivre dans cette voie.

Comme perspectives d'améliorations, nous aimerions que les mouvements via détection gant puisse se faire sur une caméra distante et non celle du drone afin de faciliter le contrôle.